

Project Overview & Task Stack;

Project Title: MicroPad – Text Editor:

Week 1: Project Setup, Git & UI Base Structure

- **Focus:** Initializing the environment, connecting Git, and creating the main application window shell.
- **Tasks:**
 - **Git Setup:** Initialize a local Git repository and connect it to a GitHub remote repo. Set up main and development branches.
 - **Environment Setup:** Create a JavaFX project and configure the UI/UX wrapper (WebView and WebEngine components).
 - **Base Structure Layout:** Build the main application window shell.
 - **Empty Menu Bar:** Design and display a top menu bar containing just the placeholder headings: **File**, **Edit**, and **Help** (no sub-menus or logic yet).
 - **Git Milestone:** Commit the empty shell structure to feature/base-ui and push it to GitHub.

Week 2: File Menu Architecture & Sub-Items (Core Setup)

- **Focus:** Populating the File menu with sub-items and starting Java-side file handling.
- **Tasks:**
 - Add sub-menu items under **File**: New File, Open File, Save, Save As, and Exit.
 - Integrate the Java FileChooser component to navigate the native local system directory.
 - Write the basic Java background methods required to capture a selected file's path.

Week 3: File I/O Logic Implementation (Reading & Saving)

- **Focus:** Making the File menu fully operational by reading and writing files.
- **Tasks:**
 - Develop Java backend logic to read text content from .txt, .java, or .md files and stream it into the editor UI.
 - Implement data stream writing logic for **Save** and **Save As** actions.

- Add alert handling to prompt users to save changes if they attempt to **Exit** without saving.

Week 4: Edit Menu Sub-Items & Clipboard Logic

- **Focus:** Building out the Edit menu features.
- **Tasks:**
 - Add sub-menu items under **Edit**: Undo, Redo, Cut, Copy, and Paste.
 - Establish a JavaScript-to-Java Bridge (JSObject) so UI menu click events trigger backend Java controllers.
 - Bind native system clipboard shortcuts (Ctrl+C, Ctrl+V, etc.) to the text editor field.

Week 5: Advanced Editing Operations (Undo/Redo Engine)

- **Focus:** Managing history states for text modifications.
- **Tasks:**
 - Implement an Undo/Redo tracking engine using Java Data Structures (like Stack) to log keystroke events.
 - Link the **Edit -> Undo** and **Edit -> Redo** buttons directly to this Java tracking engine to revert or re-apply changes.

Week 6: Help Menu & Theme Customization (UI/UX)

- **Focus:** Completing the Help section and introducing app themes.
- **Tasks:**
 - Add sub-menu items under **Help**: About Editor and Documentation.
 - Create a clean pop-up modal dialog for the About screen containing project metadata and developer credits.
 - Design a Dark Mode and Light Mode styling option, accessible via a small toggle switch or menu button.

Week 7: Core Feature expansion - Find & Replace

- **Focus:** Building a robust tool to navigate large files efficiently.
- **Tasks:**
 - Design a collapsible search bar or panel in the UI overlay.

- Write Java logic using Regular Expressions (Regex) to search for specific terms and support Replace and Replace All text modifications.
- Add text highlighting for search matches.

Week 8: Multi-File Support (Tabbed Interface)

- **Focus:** Transitioning from a single-file editor to a browser-like tabbed workspace.
- **Tasks:**
 - Design and build a dynamic modern Tab interface layout.
 - Create a Java Object-Oriented system to isolate file paths, contents, and tracking instances for every newly opened tab.
 - Manage tab switching and handle confirmation dialogs for closing individual tabs with unsaved data.

Week 9: Live Git Status Tracker (In-App Integration)

- **Focus:** Providing direct visibility into version control states inside the app layout.
- **Tasks:**
 - Connect the application to local Git instances using Java's ProcessBuilder or the JGit library.
 - Build a bottom Status Bar to show real-time metrics: Current line/column index, word counts, and the file's current Git state (e.g., Modified or Untracked).

Week 10: Optimization, Deployment & Release

- **Focus:** Code cleanup, performance testing, and packaging the standalone tool.
- **Tasks:**
 - Utilize Java background threading (Platform.runLater) to keep the interface smooth and lag-free when handling very large files.
 - Package the Java application into an executable .jar or standalone platform installer (.exe / .dmg).
 - Write a complete README.md on GitHub, clean up commit histories, merge the final working code into the main branch, and tag the release as v1.0.